

MASTER ARCHITECTURE & MATHEMATICAL SPECIFICATION

Architecture Principle: To guarantee mathematical stability and a sub-2.0s response SLA, the mobile application acts purely as a presentation layer. The Node.js gateway handles spatial graph mapping (Valhalla), while a dedicated FastAPI Python microservice executes all electrochemical thermodynamics (FASTSim) and stochastic probability inferences (DistilBERT).

1. Global Parameter Provenance & Ingestion Matrix

To preserve a zero-friction user experience, the ingestion matrix separates explicit user inputs from automated, out-of-band telemetry lookups.

Parameter Symbol	Sourcing Mechanism	Operational Target Function
Origin / Destination	Mobile UI Geocoding API	Establishes the geometric bounding box for PostGIS queries and Valhalla routing.
Vehicle ID String	User Account Profile	Acts as the relational foreign key to query FASTSim structural constraints from the database.
Initial SOC (%)	Interactive App Slider	Sets the boundary state condition vector $SOC[0]$ for the physical simulation sequence.
Ambient Temp (T)	Tomorrow.io Real-Time API	Injected into the NASA 2nd-order Arrhenius polynomial to adjust instant battery capabilities.
Road Grade (θ)	Valhalla SRTM 30m Tiles	Determines second-by-second gravitational loads during the forward physics cycle.
C_{base}, C_d, A, M	PostgreSQL Core Storage	Physical constants (Capacity, Drag, Frontal Area, Weight) required to instantiate the vehicle object.
Driver Reviews Logs	Internal DB + Public RSS	Feeds unstructured text strings to the DistilBERT pipeline to determine risk probabilities.

2. Phase 2: Python Physics Bridge Contract

Node.js gathers the enriched telemetry and executes an internal HTTP POST request to the Python worker. The payload strictly defines the physical constants and the macro-scale road geometry.

2.1 Internal Payload Schema (Node.js → Python)

```
{
  "vehicle_dictionary": {
    "maxEssKwh": 40.50, "vehCgM": 1400.0, "maxMotorKw": 106.0,
    "dragCoef": 0.33, "frontalAreaM2": 2.40
  },
  "runtime_conditions": {
    "ambient_temp_c": -12.5, "state_of_health": 0.88, "starting_soc": 0.82
  },
  "valhalla_edges": [
    { "distance_m": 1200.0, "speed_kph": 100.0, "grade_pct": 2.50, "start_lat": 43.653, "start_lon": -79.1
  ]
}
```

2.2 Edge-to-Cycle Granular Interpolation

Because NREL's FASTSim requires sequential 1Hz data streams, the Python bridge decomposes macro-scale parameters into second-by-second intervals. The duration for any segment is derived via:

$$Duration(seconds) = \text{round} \left(\frac{\text{distance_m}}{\text{speed_kph}/3.6} \right)$$

2.3 The Depletion Pointer Return

After executing `SimDrive.sim_drive()`, the script linearly sweeps the output array to find the exact geographical coordinate where the SOC drops below the 10.0% safety threshold. This coordinate is returned to Node.js to trigger the PostGIS spatial charger search.

3. Phase 3: Electrochemical Capacity Calibration (NASA PCoE)

To prevent the physics simulator from overestimating range, the factory battery capacity (C_{base}) is mathematically degraded using polynomial regression derived from the NASA Ames Prognostics Center of Excellence (PCoE) datasets ($B0005 - B0018$).

3.1 State of Health & Thermal Math

State of Health tracks continuous cycle aging using a power-law equation, while a second-order Arrhenius polynomial scales capacity based on ambient temperature (T):

$$S_{OH} = 1 - \alpha \cdot N^\beta$$

$$\Phi_{temp}(T) = aT^2 + bT + c$$

Where empirically derived constants are bound to $a = -1.14 \times 10^{-4}$, $b = 5.72 \times 10^{-3}$, and $c = 0.924$. The corrected chemical storage capacity injected into FASTSim is evaluated as:

$$C_{eff} = C_{base} \cdot S_{OH} \cdot \Phi_{temp}(T)$$

4. Phase 3: Merged NLP Ingestion & Semantic Scoring

Our routing algorithm relies on an accurate P_{fail} score. To calculate this, a nightly asynchronous Celery worker ingests unstructured text from two distinct sources to evaluate hardware reliability.

4.1 Hybrid Data Sourcing & Trust Weighting (τ)

Not all data sources are equally reliable. The system ingests data from two pipelines and applies a **Source Trust Scalar** (τ) to prevent anonymous internet trolling from outweighing our verified users.

1. **Source A: Proprietary Review Database (High Trust):** Queried directly from our internal `station_reviews` table. Because these reviews are linked to authenticated user accounts who physically checked in, this source is assigned $\tau_{internal} = 1.0$.
2. **Source B: Public RSS Scraping (Medium Trust):** Scraped from public forums like `r/evdrivers` and PlugShare. The worker uses BERT-NER to map mentions to specific `station_id` s. Due to anonymity, this source is assigned $\tau_{public} = 0.65$.

4.2 NLP Semantic Normalization

The `review_text` is passed to the HuggingFace `DistilBERT-SST-2` model. The resulting positive sentiment logits are processed through a Softmax function (s_i). We invert this to calculate the base Sentiment Penalty:

$$X_{\text{sentiment},i} = 1.0 - \text{Softmax}(z_{pos})$$

4.3 Trust-Weighted Temporal Decay

Hardware gets repaired over time, necessitating an exponential decay algorithm. We calculate the age of the review in days (Δt_i) and apply a 30-day half-life decay constant ($\lambda \approx 0.0231$). We fuse this temporal decay with the Source Trust Scalar (τ) to yield the final review weight (w_i):

$$w_i = \tau_i \cdot e^{-\lambda \cdot \Delta t_i}$$

The consolidated historical NLP constraint ($\hat{S}$) for a charging node is the weighted arithmetic mean of all M reviews:

$$\hat{S} = \frac{\sum_{i=1}^M (w_i \cdot X_{\text{sentiment},i})}{\sum_{i=1}^M w_i}$$

5. Logistic Aggregation & Stochastic Pathfinding

5.1 The Multivariate Logistic Engine

The worker consolidates the sentiment constraint ($\hat{S}$), real-time OCPI API heartbeat ($X_{api} \in \{0, 1\}$), and physical hardware age (X_{age}) into a Logistic Regression logit vector (z). Using our empirical coefficients (β_n), we compute:

$$z = \beta_0 + (\beta_1 \cdot X_{api}) + (\beta_2 \cdot \hat{S}) + (\beta_3 \cdot X_{age})$$

To map this boundless logit to a strict probability fraction ($0.0 \rightarrow 1.0$), we apply the Sigmoid activation function to define the final Probability of Failure (P_{fail}):

$$P_{fail} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

This Float32 value is serialized directly into the Redis cluster (e.g., `SET charger:EC_10492:pfail 0.1425`) for sub-millisecond retrieval by the Node.js application.

5.2 Expected Journey Time ($E[T]$) Optimization

When the FASTSim physics engine triggers a reroute, Node.js queries PostGIS for surrounding chargers. It reads their P_{fail} scores from Redis and mathematically penalizes risky stations using the Stochastic Expected Cost equation:

$$E[T] = (1 - P_{fail}) \cdot T_{optimal} + P_{fail} \cdot T_{penalty}$$

Routing Implication: If an ultra-fast charger has an optimal stop time ($T_{optimal}$) of 20 minutes, but a high failure probability ($P_{fail} = 0.50$) yielding a 90-minute worst-case detour ($T_{penalty}$), its expected graph cost $E[T]$ inflates to 55 minutes. Valhalla will automatically bypass it in favor of a slower, but 99% reliable, backup charger.